PERIFERIA IT GROUP · RETO TÉCNICO 01

Documentación funcional y de arquitectura

Agente conversacional Registro como Proveedor

Cómo está organizado el proyecto y cómo funciona por dentro, explicado de forma clara.

TypeScript · Node 20+ · Agente + Herramientas + LLM intercambiable

Documento de comprensión del proyecto

1 ¿Qué resuelve este proyecto?

> Cuando un cliente quiere registrar a Periferia como su proveedor, le envía un formulario para llenar y le pide adjuntar soportes. Hoy una persona copia esos datos a mano. Este proyecto es un agente que lo hace solo.

El agente, a partir de una solicitud recibida por correo:

1. **Lee** la solicitud y entiende qué campos y soportes pide.
2. **Busca** cada dato en el repositorio maestro de Periferia.
3. **Llena** el formulario en el mismo formato que mandó el cliente (Excel, PDF o portal).
4. **Arma** un paquete para firma: formulario + soportes + checklist + borrador de correo.
5. **Avisa** qué falta o qué confirmar, y **nunca envía ni firma sin autorización humana**.

Tú le hablas por un chat (*"procesa el caso X"*) y él va mostrando cada paso.

2 Las tres ideas que explican todo

01

El agente no inventa datos

El modelo no conoce el NIT ni la cuenta. Esos valores solo salen de las **herramientas**. El agente decide cuándo usarlas; el dato siempre viene de ellas.

02

Tres responsabilidades separadas

Comportamiento en el prompt, **conocimiento** en knowledge, **ejecución** en tools. Cambiar una regla no toca el motor.

03

El modelo es intercambiable

Funciona con un modelo falso (mock, sin clave) o uno real (OpenAI, Anthropic). Cambiarlo es cambiar una variable de entorno.

3 Distribución de carpetas

```
reto-01/
├── agent/prompt.md          → El "instructivo" del agente (personalidad y reglas)
│
├── src/                     → Todo el código del backend
│   ├── server.ts            → Levanta el servidor web y la API
│   ├── agent/               → EL MOTOR
│   │   ├── loop.ts          → El ciclo: modelo → herramienta → repite
│   │   └── sessions.ts      → Memoria de cada conversación (en RAM)
│   ├── llm/                 → CONEXIÓN AL MODELO
│   │   ├── adapter.ts       → El "enchufe" estándar
│   │   ├── mock.ts          → Modelo falso, sin clave (por defecto)
│   │   ├── openai.ts        → Conexión real a OpenAI
│   │   ├── anthropic.ts     → Conexión real a Claude
│   │   └── factory.ts       → Elige cuál usar
│   ├── tools/               → LAS HERRAMIENTAS (el trabajo real)
│   │   ├── proveedor.ts     → Define las 5 herramientas del agente
│   │   ├── registry.ts      → Las registra y valida argumentos
│   │   ├── fixtures-io.ts   → Lee los archivos de entrada
│   │   ├── mapeo.ts         → Cruza cada campo con el maestro
│   │   ├── generadores.ts   → Crea el Excel / PDF / portal
│   │   └── paquete.ts       → Arma el paquete para firma
│   ├── domain/              → PIEZAS BASE compartidas
│   │   ├── types.ts         → Los "moldes" de datos
│   │   ├── reglas.ts        → Reglas puras (país, vigencia, confianza)
│   │   ├── maestro.ts       → Cómo leer un dato del maestro
│   │   ├── paths.ts         → Todas las rutas en un solo lugar
│   │   └── logger.ts        → Registro de lo que hace cada herramienta
│   └── knowledge/           → Las reglas del proceso, en texto
│
├── web/                     → EL CHAT (navegador): html + css + js
├── fixtures/                → DATOS DE ENTRADA (los da Periferia, NO se tocan)
├── out/                     → RESULTADOS generados (se puede borrar)
├── modulo/                  → BONUS: agente empaquetado para reusar
├── demo.ts                  → Prueba las herramientas sin modelo
├── README.md · SOLUCION.md · DOCUMENTACION.md
```

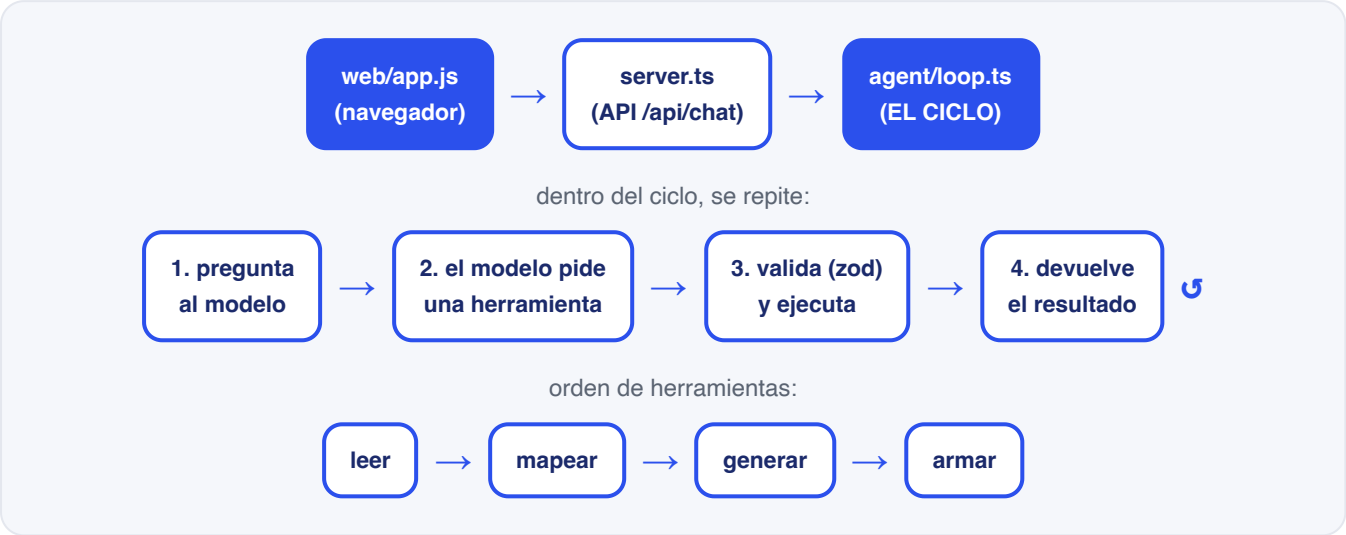
Jerarquía de dependencias (de abajo hacia arriba)

domain/	piezas base, no dependen de nada
tools/	usan domain para hacer el trabajo real
llm/	el "enchufe" del modelo, independiente
agent/	usa tools + llm: es el motor del ciclo
server.ts	usa agent y sirve el chat web

Las capas de abajo no saben que existen las de arriba. Eso hace el código fácil de probar y de cambiar por partes.

4 ¿Cómo funciona por dentro?

Supón que escribes: *"Procesa el caso ec-corp-andina. No envíes nada."*



El ciclo se repite (modelo → herramienta → modelo…) hasta que el modelo tiene todo y responde con texto. Hay un **tope de 25 vueltas** para que nunca se quede dando vueltas infinitas. Al final devuelve `{ reply, toolCalls, needsConfirmation }` y el chat muestra el resumen, cada herramienta usada y, si aplica, el aviso de confirmación.

5 Las 5 herramientas

El agente solo puede hacer lo que estas herramientas permiten. Cada una valida sus argumentos y devuelve siempre `{ ok, data }` o `{ ok:false, error }` — nunca "explota".

Herramienta	Qué hace	Resultado
`leer_solicitud`	Abre la solicitud del cliente y su plantilla	País, cliente, formato, campos y soportes pedidos
`mapear_campos`	Cruza cada campo con el maestro usando el glosario	Campos llenos, faltantes y a confirmar
`generar_formulario`	Crea el formulario en el formato pedido	`.xlsx`, `.pdf` o `valores-portal.md`
`armar_paquete`	Copia soportes, arma checklist y borrador de correo	Carpeta `paquete/` + si está listo para firma
`simular_envio`	Simula el envío (solo con confirmación humana)	`ENVIO-SIMULADO.md` o error

Los tres estados de un campo

LLENO	Lo encontró con certeza. Se escribe el valor.
FALTANTE	No existe en el maestro. **No se inventa.** Se reporta.
REQUIERE CONFIRMACIÓN	Lo encontró pero hay una duda (ej. cliente de Ecuador pide "RUC", Periferia solo tiene NIT). Se llena y se marca para que un humano confirme.

Qué bloquea la firma

El paquete queda **"listo para firma"** solo si todos los soportes exigidos están presentes y vigentes. Lo bloquea un soporte **ausente** o uno **vencido**. Un campo faltante no bloquea la firma, pero sí aparece en el checklist.

6 Los datos de entrada (fixtures)

Son los archivos que simula Periferia. **No se modifican**; el agente solo los lee.

Archivo	Qué es
`casos/<caso>/solicitud.json`	El correo del cliente: quién es, país, formato
`casos/<caso>/plantilla-celdas.json`	(Excel) qué va en cada celda
`casos/<caso>/plantilla-campos.json`	(PDF) la lista de campos
`casos/<caso>/soportes-exigidos.json`	Qué soportes pide ese cliente
`repositorio/maestro.json`	**La ficha de Periferia** (NIT, dirección, banco…)
`repositorio/soportes/`	Los soportes + su índice con vigencias
`glosario-campos.json`	Diccionario: "Nombre de la empresa" → `razon_social`

Los 4 casos de ejemplo

Caso	Formato	Qué prueba
co-industrias-delta	Excel	Caso ideal, todo se llena · LISTO
ec-corp-andina	PDF	Pide RUC (confirmar) y le falta un soporte · BLOQUEADO
hn-agroexport-sula	Excel	Pide RTN (confirmar) y un soporte vencido · BLOQUEADO
pa-logistica-istmo	Portal	Formato no soportado, genera valores para copiar

7 Cómo probarlo

Opción rápida: la demo (sin navegador, sin clave)

```
npm install
npm run demo
```

Procesa los 4 casos de corrido y muestra un resumen de cada uno. Los resultados quedan en `out/`.

Opción completa: el chat

```
npm run dev
```

Abre `http://localhost:3001` y escribe, por ejemplo: *"Procesa el caso ec-corp-andina. No envíes nada todavía."* Verás el resumen, cada herramienta que usó y un aviso si espera tu confirmación.

8 Glosario rápido

Agente	El programa que conversa, decide qué herramientas usar y responde.
Herramienta (tool)	Función concreta que hace una tarea y devuelve datos. Única fuente de valores reales.
Ciclo del agente	El bucle "pregunto al modelo → ejecuto herramienta → repito".
Adaptador LLM	La pieza que habla con el modelo (mock, OpenAI o Anthropic).
Maestro	La ficha con todos los datos de Periferia.
Fixture	Dato de entrada de ejemplo que da Periferia.
zod	Librería que valida la forma de los argumentos antes de ejecutar una herramienta.
Confirmación humana	El agente se detiene y pregunta antes de cualquier acción externa.